

Response to the Editorial Return

exdqlm: An R Package for Estimation and Analysis of Flexible Dynamic Quantile Linear Models

Antonio Aguirre, Raquel Barata, Raquel Prado, and Bruno Sansó

Prepared June 2026

Dear Editors,

We thank the editorial team for the careful return and for the specific guidance on how to make the submission more suitable for review by the *Journal of Statistical Software*. We are submitting the revised material as a new submission, together with this point-by-point response. We revised the manuscript, package, and replication materials substantially in response. The revised submission is organized around the software contribution, uses a clearer R object and method design, and provides a standalone replication archive centered on a single reader-facing `code.R` script.

The current submission files are:

- Manuscript PDF: `exdqlm-jss.pdf`
- Software source: `exdqlm_1.1.0.tar.gz`
- Replication materials: `exdqlm-jss-replication.tar.gz`
- Point-by-point response: `response-to-editor.pdf`

The submitted package source is version 1.1.0. The package source commit used for the submission is `9b20e87933ec121a0cc88d7a0d1bdb964e87d55b`. The article source commit used for the replication archive is `be95a2874da4341464fcd61c0abfd4f2c7cba5b5`. The package is distributed under the MIT License, which is GPL-compatible.

Summary of Major Revisions

Reorganized the manuscript to make the JSS contribution explicit: prior methodology is separated from the software contribution, and a new package design and implementation section describes the object system, fitting engines, diagnostics, forecasting, plotting, and extension points.

Revised the package design for standard R use: fitted dynamic and static objects now have shared superclass families, diagnostic/forecast/ synthesis objects are visible reusable objects, and standard methods such as `print()`, `summary()`, `plot()`, and `predict()` are used where natural.

Rewrote the replication materials as a standalone JSS archive. The public workflow is now centered on `Rscript code.R`, with documented quick and single-example options. The public path no longer requires Git metadata, GitHub remotes, or internal reference-machine modes.

Merged the former supplement into the main manuscript as appendices and prepared a single manuscript PDF.

Rebuilt the final replication archive without Git metadata, local audit files, model-cache `.rds` files, scratch LaTeX files, or nested archives. We tested both a quick no-git smoke run and a full no-git replication run from a fresh archive extraction using the submitted package tarball.

1 General Editorial Comments

Comment (G1)

The submission should better indicate which parts of the methodology have already been published and what constitutes the specific contribution for Journal of Statistical Software.

Response

We agree. The revised manuscript now states more explicitly that the previously published methodological components include the exAL likelihood family and the exDQLM modeling framework, while the contribution of this article is the R software implementation and workflow. In particular, the manuscript now frames `exdqlm` as a software contribution: a user-facing R package for dynamic and static Bayesian quantile modeling, MCMC and LDVB fitting engines, deterministic diagnostics, forecasting, transfer-function state augmentation, posterior predictive synthesis, standard object methods, examples, and replication materials.

We also added a dedicated package design and implementation section before the examples. This section makes the software contribution inspectable before the reader reaches the examples, and it separates package architecture from the mathematical background and algorithmic appendices.

Comment (G2)

The replication material was split across many files and it was not clear which option should be used to re-run all computations and obtain the article results.

Response

We rewrote the replication archive around a single public entrypoint, `code.R`. The first-page replication command is now:

```
Rscript code.R.
```

Two optional commands are documented for convenience:

```
Rscript code.R -quick
```

for a short wiring/test pass, and

```
Rscript code.R -example N
```

for rerunning one manuscript example. The previous public distinction between `portable` and `reference` modes was removed from the reader-facing workflow. Developer/reference checks are kept out of the public command path.

The revised `README.md` describes the contents of the submitted archive, the package installation options, the expected outputs, and the runtime interpretation. The generated `code.html` is produced from `code.R` using `knitr::spin()` and includes `sessionInfo()`.

Comment (G3)

The package implementation needs improvement, in particular the classes and methods system. A section on package design and implementation would facilitate review.

Response

We revised both the package and the manuscript in response. On the package side, dynamic fits now inherit from a common `exdqlmFit` family, static fits from `exalStaticFit`, and post-processing outputs such as diagnostics, forecasts, forecast diagnostics, posterior-predictive synthesis, and static diagnostics have explicit classes. We expanded standard methods for these objects, including informative `print()` and `summary()` methods and `plot()` methods for diagnostic, forecast, synthesis, and coefficient summary displays. We also added `predict()` support for fitted dynamic objects where it naturally wraps forecasting.

On the manuscript side, we added a package design and implementation section that describes the object families, fitting engines, returned objects, method dispatch, backend controls, and extension points. The examples were updated to show the preferred object-first workflow.

2 Detailed Comments

Comment (D1)

The replication README referred to cloning a GitHub repository. JSS replication material should be a standalone directory without requiring unstable external GitHub repositories.

Response

We rewrote the submitted `README.md` as an archive-oriented document. It no longer asks the reader to clone or pull a GitHub repository. It begins by describing the contents of the submitted replication directory and then gives the commands needed to install the submitted package tarball or use the installed CRAN package and run the replication script. The final replication archive has also been rebuilt without `.git`, `.gitignore`, or other local repository metadata.

Comment (D2)

The submitted `code.R` launched `run_all.R`, which itself ran the different analysis parts. A master script may launch subparts, but the master script should not itself be launched by another master script.

Response

The revised `code.R` is now the public master script. It sources the manuscript setup and the example scripts directly in manuscript order rather than delegating the workflow to a second master script. Internal developer orchestration remains separate from the public reproduction path. The reader-facing script now has visible sections for setup, examples, checks, and session information.

Comment (D3)

The README advertised many different ways to launch the replication material, and the difference between `-mode portable` and `-mode reference` was unclear.

Response

We simplified the public interface. The submitted archive now documents only the full run, the quick run, and the single-example run. The previous `portable/reference` terminology is no longer part of the public replication instructions. Exact reference-machine checks remain part of our internal acceptance workflow, but they are not required for readers or editors to run the submitted replication materials.

Comment (D4)

Embedding the launched code in a `main()` function prevents users from accessing intermediate analysis results. The scripts should help readers adapt the examples to their own analyses.

Response

We removed the whole-workflow `main()` wrapper from the reader-facing `code.R`. The revised script is organized as a direct, commented replication script with small helper functions only where they clarify repeated tasks. The example scripts still write the manuscript figures, tables, logs, and provenance files to documented locations, and the public script can be sourced or run with `Rscript`. This makes the replication path easier to inspect and adapt while preserving the tested manuscript-generation workflow.

Comment (D5)

Running the documented command failed because the submitted directory was not a Git repository and the preflight checked Git remotes and branch metadata.

Response

We split the public replication path from developer/reference checks. The default submitted `code.R` workflow no longer requires Git metadata, Git remotes, branch freshness, or a GitHub checkout. We verified this by extracting the final replication archive into a fresh directory with no `.git` metadata and running:

```
EXDQLM_LOAD_MODE=installed Rscript code.R -quick.
```

This quick no-git smoke test passed. We also ran the full no-git replication from a fresh extraction using the submitted `exdqlm_1.1.0.tar.gz` package tarball installed into a staging library. The full run completed, regenerated the manuscript outputs, rebuilt the manifest, and completed the manuscript tests.

Comment (D6)

The manuscript lacks a section describing the package and its design, including the methods provided, implementation, possible extensions, and variants.

Response

We added a package design and implementation section before the examples. This section describes the principal object families, class inheritance, fitting engines, diagnostic and forecast objects, synthesis objects, plotting and post-processing methods, backend controls, and extension points. It also clarifies why some operations remain named workflow functions while returned objects use standard R methods for printing, summarizing, plotting, and prediction.

Comment (D7)

It would be preferable to have different, more informative print and summary methods for `exdqlm` objects.

Response

We audited the package object families and revised the standard display methods. Representative fit, diagnostic, forecast, forecast-diagnostic, synthesis, and static-diagnostic objects now have informative `print()` and/or `summary()` output indicating the object type and key dimensions or workflow quantities. The package test suite includes coverage for these standard methods.

Comment (D8)

It is unclear why the package does not make use of class inheritance. Fitted models could share a generic class with common methods and variant-specific additions.

Response

We introduced shared superclass families while preserving existing first-class names for backward compatibility. Dynamic fitted objects now inherit from `exdqlmFit`; static AL/exAL fitted objects inherit from `exalStaticFit`; and reusable post-processing objects inherit from appropriate diagnostic, forecast, synthesis, or static-diagnostic classes. This lets shared methods dispatch on common fit families while preserving variant-specific behavior where needed.

Comment (D9)

It is unclear why the post-processing functions are not methods. This would seem more natural with a suitable class and method system.

Response

We revised the post-processing workflow to be more object-centered. Operations that naturally display, summarize, plot, or predict from an existing object now use standard methods where appropriate. For example, fitted dynamic objects can be used with `plot()` and `predict()`, diagnostics and forecast diagnostics return objects that can be printed, summarized, and plotted, and the static diagnostic coefficient plots are produced through the returned diagnostic object. We kept named workflow functions for operations that construct new analysis objects from substantial additional inputs, such as diagnostics from a fit, held-out forecast diagnostics, and posterior-predictive synthesis across separately fitted quantile levels. The new design section explains this division.

Comment (D10)

The diagnostic workflow should return the object in a normal non-invisible way, with plotting handled by a `plot()` method rather than by a plotting argument.

Response

We changed the diagnostic workflow to be object-first. Dynamic diagnostics now return a visible diagnostic object, and the manuscript examples use the step-wise pattern:

```
diag <- exdqlmDiagnostics(fit)
```

followed by `summary(diag)` or `plot(diag)`. This same object-first style is used for forecast diagnostics and static diagnostics where applicable. Compatibility plotting arguments are no longer the primary workflow shown to users.

Comment (D11)

A single PDF file should be submitted as the manuscript. Supplementary material should be included directly as appendices. JSS has no page limits.

Response

We merged the former supplement into the main manuscript as appendices and are submitting one manuscript PDF, `exdqlm-jss.pdf`. The obsolete standalone supplement PDF and separate supplement bibliography artifact were removed from the submission bundle. The appendix source remains as a LaTeX input file for maintainability, but it is compiled into the single manuscript PDF.

3 Validation and Final Submission Checks

All final checks below were run with R 4.6.0.

- R CMD check `-no-manual -run-donttest exdqlm_1.1.0.tar.gz` completed with 0 errors, 0 warnings, and 0 notes.
- The article reproducibility preflight completed with 0 errors and 0 warnings.
- The final replication archive audit found no Git metadata, ignored audit documents, `.rds` caches, obsolete supplement PDF, obsolete supplement bibliography artifact, LaTeX scratch files, or nested archives.
- A no-git quick smoke test from the extracted replication archive passed using the installed package from the submitted package tarball.
- A full no-git replication run from a fresh extraction completed successfully and regenerated manuscript outputs from absent caches.

We believe these revisions address the editorial return and make the submission substantially easier to inspect, reproduce, and review as a JSS software article.